The first line of the trace is `#0 /usr/src/myapp/call_chains.php(14): c()` – this is an instruction to the programmer to look at line 14 of the `call_chains.php` file because that's where the call to `c()` is found. The next line shows us that `c()` was called because `b()` was executing at line 6 of `call_chains.php`… and so on. Read from top to bottom, we can decipher it like this:

- Exception thrown, line 22 of `call_chain.php`
- The reason for this is because `c()` was called at line 14…
- … because `b()` was called at line 6…
- … because `b()` was called at line 25, and…
- … now we're at the main program (no more calls)

We didn't need a stack trace to understand why our code exploded; we literally have all three functions on screen at the same time (unless your terminal or IDE font size is unusually big, and even then all the calls are in the same file), but in the real world, you might struggle to juggle a stack that's a couple of dozen calls deep across as many files. Traces like this become invaluable debugging tools when projects are spread across hundreds of files, with hundreds of functions and/or objects.

`debug_backtrace()`, `debug_print_backtrace()`

You can also trigger your own stack traces if necessary; debugging doesn't have to be about fixing fatal errors. You might simply want to investigate the internals of a new library or framework. Want to know what happens before your brilliant new component's entry-point method is invoked by ShinyFramework? A trace can show you all the calls made from the framework booting up before the code that you wrote ever ran. They can be a great tool for learning. A few well-placed calls to PHP's `debug_backtrace()` or `debug_print_backtrace()` functions will generate a stack trace at the point where they are called. Calling either of these functions does not affect the flow of the program; it will resume immediately after the output.

`debug_backtrace()` will return a structured array with an element for each entry in the call stack; `debug_print_backtrace()` produces the same trace structure but then converts it into a formatted string (to save you having to write your own loops to render). Modify the previous example so that the implementation of `function c()` is as follows…